Cleisson de Oliveira Moura

Implementation Planning vs. Developing: A Mental Model I Keep Coming Back To

A practical reflection on how implementation planning can reduce development effort, without pretending the chart is a scientific model.

Published: June 9, 2026 · Updated: September 5, 2026

Source article: https://www.cleisson.com/en-US/blog/implementation-planning-vs-developing

On the project I work on, we started creating separate tasks for implementation plans. Sometimes the development task would only be created, refined, or broken down after the planning task was finished.

At first, I wanted to jump straight into the code. That is where things feel concrete. But after seeing this process a few times, I came to appreciate having time to understand what a change would touch before committing to an approach. In features, refactors, integrations, and migrations, that understanding can take more work than the ticket suggests.

My working rule is to plan while it is the cheaper way to answer a question that could change the implementation. Once a small, reversible experiment would teach us more, it is time to build. I judge the plan by the effort and risk it saves across the whole change, including the time spent planning.

By implementation planning, I mean reading existing code, understanding system boundaries, checking assumptions, and working out the structure, tests, and manual steps a change needs. Developing covers writing code, adjusting tests, reviewing, integrating, debugging, deploying, and handling what we discover along the way.

These activities overlap. A small implementation spike can be part of planning, and working code can invalidate a plan. Separate tickets help organize the work; they do not make learning happen in a straight line.

Why a separate planning task can help

Without an explicit planning step, investigation often gets scattered across ticket comments, Slack threads, and discoveries made during development. For a small change, that may be enough. For a larger one, I want those findings somewhere the team can use them before work depends on an untested assumption.

A dedicated task makes that investigation visible. It gives an engineer time to read the code, ask who owns a flow, and check whether an existing pattern solves part of the problem. The result might be a short note with affected areas, an implementation sequence, and risks the team needs to discuss.

The development task that follows can then have clearer boundaries and test expectations. It can also include the boring manual steps that are easy to leave out until release day.

I would scale the plan to the consequences of getting the change wrong. A local change that is easy to undo may need only a note in the ticket. A migration with dependencies and legacy behavior deserves closer investigation. A separate planning task is useful when it gives that work room to happen; I would not make it a requirement for every code change.

What I want a plan to uncover

A ticket rarely contains the whole history of a system. Business rules, old compromises, and areas marked *"please do not touch this unless you know why"* can all change the approach.

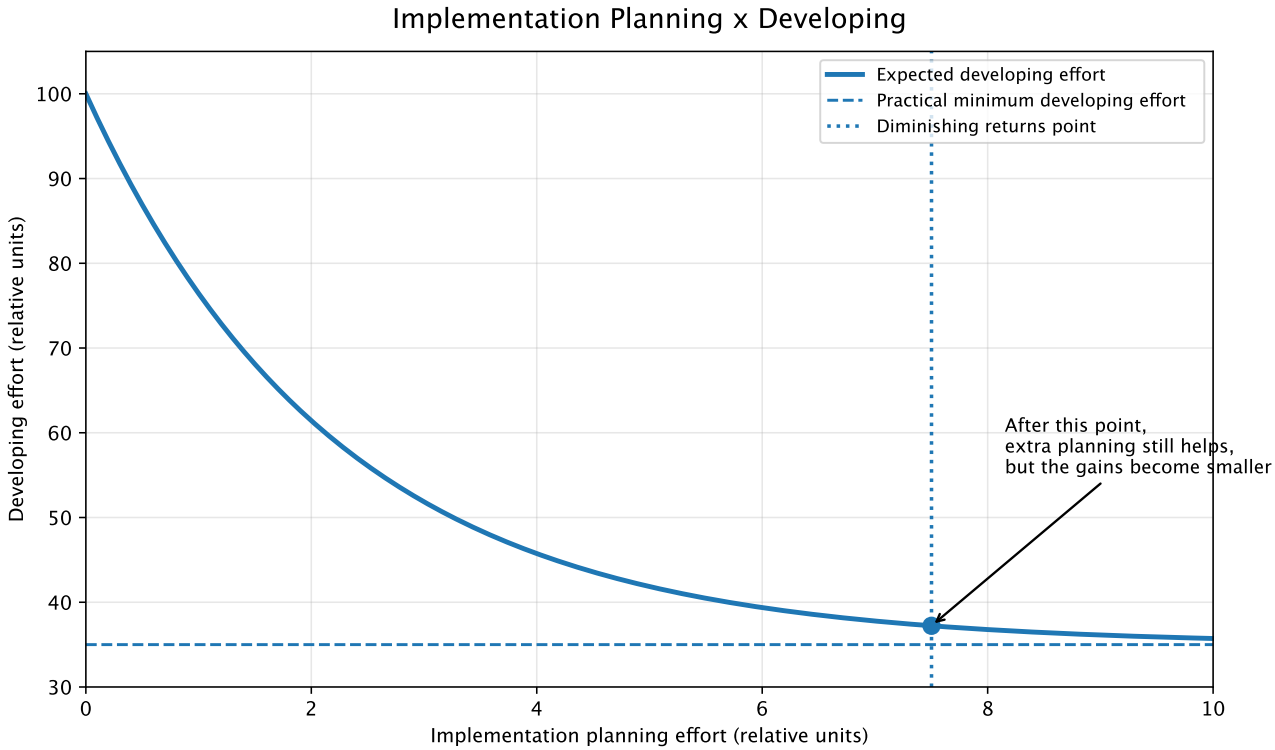
I want the plan to answer a few practical questions:

- What is changing, what is out of scope, and which systems, modules, jobs, events, or APIs are affected? Who owns the flows involved?
- Which business rules and legacy behaviors must survive? Which existing code patterns should we follow or avoid?
- What structure and implementation sequence make sense, and which assumptions could force us to change them?
- Which tests and edge cases will tell us whether the change works? What can we automate, and what still needs human verification?
- What has to happen around release, including scripts, migrations, backfills, feature flags, configuration, and deployment order? Who owns the manual steps, what do we check afterward, and how do we recover if something goes wrong?

The answers can fit in a ticket. For a complicated change, some may need their own investigation. I care about whether they help the next engineer make a decision, rather than how much documentation they produce.

What the curve gets right

This is how I picture the relationship between useful planning and the development effort that follows:



Illustrative model — based on practical observation, not on formal data analysis.

The units are illustrative. This is a mental model drawn from experience, not a study or a benchmark.

The steep early drop represents the avoidable work a short investigation can expose. We might discover that a flow already exists, a field has a hidden business meaning, or a dependency behaves differently in production. Learning why a similar change caused problems can alter the approach before we have much code to undo. An hour spent finding an existing solution can spare hours of implementing a replacement.

That is the kind of saving I have in mind: fewer late discoveries, less back-and-forth over ownership, and less *"oh, this also affects that other service."*

But some work merely moves earlier. Reading a module during planning instead of during development still takes time. A shorter development task does not, by itself, show that the team saved effort. The saving comes when that earlier understanding avoids work we would otherwise have to redo, or helps us choose a simpler implementation.

Eventually the curve flattens. Someone still has to write, test, review, integrate, and deploy the change. Some questions also need working code before we can answer them. A longer document can make us feel more certain without resolving those questions.

The cost missing from the chart

One way to sketch the curve is the equation I keep coming back to:

```
Developing(P) = D_min + (D0 - D_min) * e^(-kP)
```

Here, `P` is planning effort and `Developing(P)` is the expected development effort afterward. `D0` is development effort with no planning, while `D_min` is the practical minimum that remains even after useful planning. The parameter `k` controls how quickly the curve approaches that minimum; it stands in for how strongly planning reduces uncertainty and rework.

With `D0 > D_min` and `k > 0`, the curve always slopes down. Looking only at that curve makes more planning look better indefinitely. But planning has a cost too.

For a fixed scope, with planning and development measured in the same effort units, the fuller picture is `Total(P) = P + Developing(P)`. Each activity belongs on one side of that sum, so an exploratory spike counted as planning should not also be counted as development.

In this simplified model, another hour of planning reduces total effort only if it saves more than an hour downstream. Once the saving falls below that, additional planning increases the total even though the development estimate keeps shrinking. For a well-understood change, extra planning may not pay for itself at all. That gives the flattening curve a practical consequence: we need a reason to keep planning.

I would not use this equation to estimate a sprint or calculate a planning percentage. I have no measured values for these parameters. It assumes useful planning and a fixed scope, while actual planning can uncover missing work and make an estimate grow. That discovery may prevent an incomplete release; a smaller estimate would have been misleading.

Effort is only part of the decision, too. For a change with serious failure consequences, I would spend more time validating recovery even if it did not shorten the implementation. A plan still has to meet the safety and correctness needs of the change.

Choose the next step by what it can teach you

Consider a hypothetical field migration. Before changing the schema, I would want to know which jobs and services depend on the field, which legacy behavior must survive, and what deployment order would keep those consumers working. Those answers could change the implementation sequence.

Now suppose the remaining question is how a backfill will behave on representative data. A bounded test in a safe environment may teach us more than another design discussion. The plan can specify what to test and what result would force us to reconsider the approach. Then we can run the experiment and revise the plan.

The same change can need more investigation in one area and working code in another. I would spend more planning effort on a decision that is hard to reverse, and use small experiments where we can learn without committing the rest of the system.

This is also why I would revisit the plan during development. When implementation exposes a wrong assumption, updating the plan helps the next task use what we have learned. Continuing with the original sequence just because the planning ticket is closed would defeat the purpose.

How I decide we have planned enough

I have both under-planned and over-planned. The pressure to move fast can leave basic questions about impact, tests, or rollout to interrupt development. The desire to avoid mistakes can keep us asking a document for answers that need an experiment.

For me, the sweet spot is still somewhere between chaos and theater. I want the team to be able to name the next bounded change, explain why it is a sensible place to start, and describe how we will check it. The main risks and manual release steps should be visible, even when some details still need validation.

The remaining unknowns should be explicit. We need to distinguish what must be resolved before release from what we can safely learn during implementation, and agree on what would make us stop or change direction. That is more useful than asking everyone whether they feel confident.

Before spending more time on a plan, I want to ask: *"What decision will the next planning step change, and why is planning the best way to answer that question?"*

When we can name an expensive assumption and a useful way to check it, keep planning. When the next answer needs working code, build the smallest part that can give us that answer and bring what we learn back into the plan.